

Intelligente Informationssysteme

Block 4:

Agents – Multi-Agent-Problem-Solver – Agentic AI

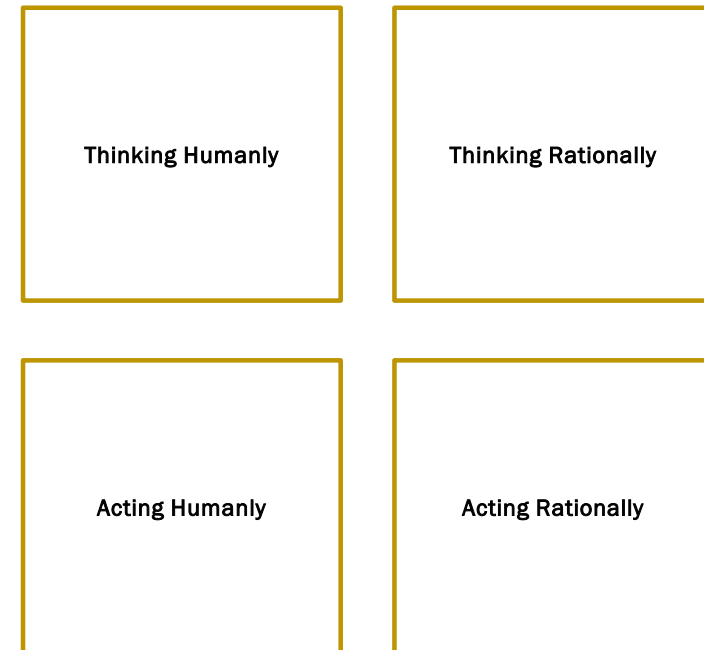
Dominik Neumann



RECAP: Artificial Intelligence - A Modern Approach

Artificial Intelligence is

- technology that enables computers and machines to **simulate human intelligence** and **problem-solving capabilities**.
- the ability of machines to think, analyze, learn and decide in **a rational way** that is **analogous to how human beings do**.
- refers to systems that **display intelligent behavior** by analyzing their environment and taking actions – with some degree of autonomy – to achieve specific goals.



To determine whether a system is intelligent,
we have to observe it.



RECAP: Compound AI Systems – Design Patterns

Conversational AI

Simulate a conversation with the feeling of having a conversation with a human.

- conversational memory
- dialogue generation

Retrieval Augmented Generation

Knowledge Retrieval and understanding is key

- Access to contextual data
- Retrieval and Augmentation Strategies

CoPilot

Assists a human in his work.

The key differentiator for becoming a CoPilot is understanding of the environment in which the human works.

- access to tools and data,
- reasoning and planning capabilities,
- and specialized profiles

Multi Agent Problem Solver

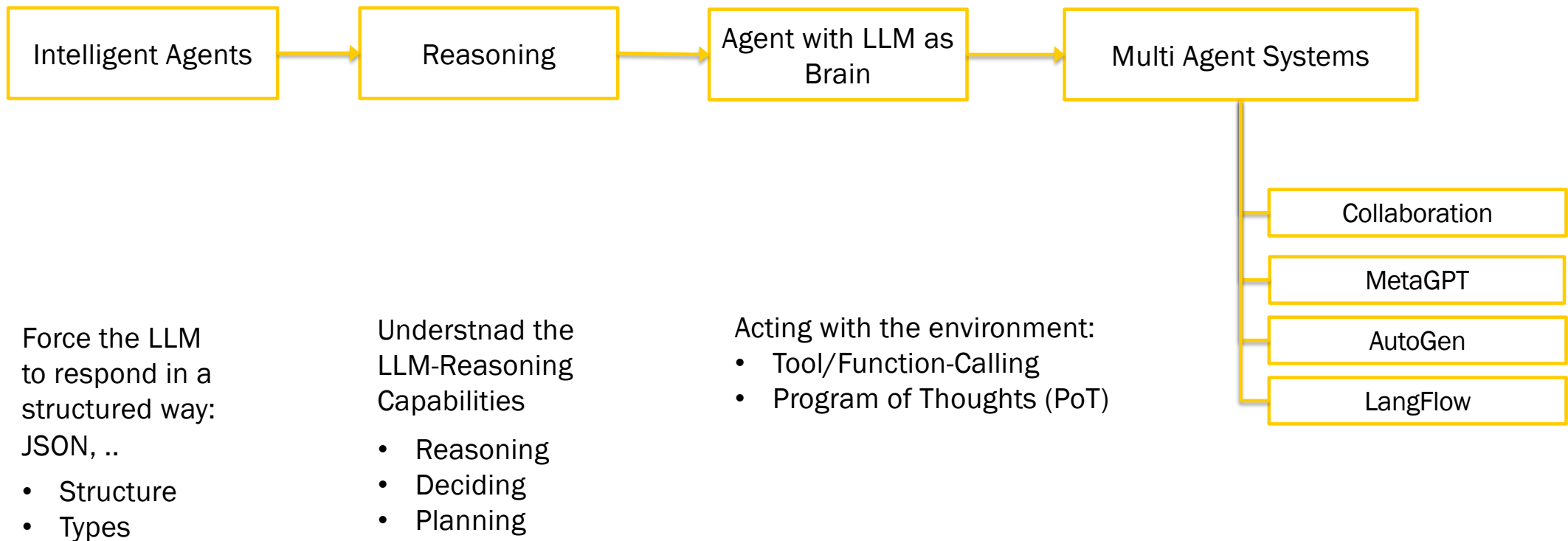
Agents collaborate to solve a problem

Each agent has access to it's own set of tools and can assume a very specific role while reasoning and planning it's actions.

- Perceiving the environment
- Reasoning & decision making
- Acting



Our Way from LLMs to Multi-Agent-Problem Solver



01 Intelligent Agents



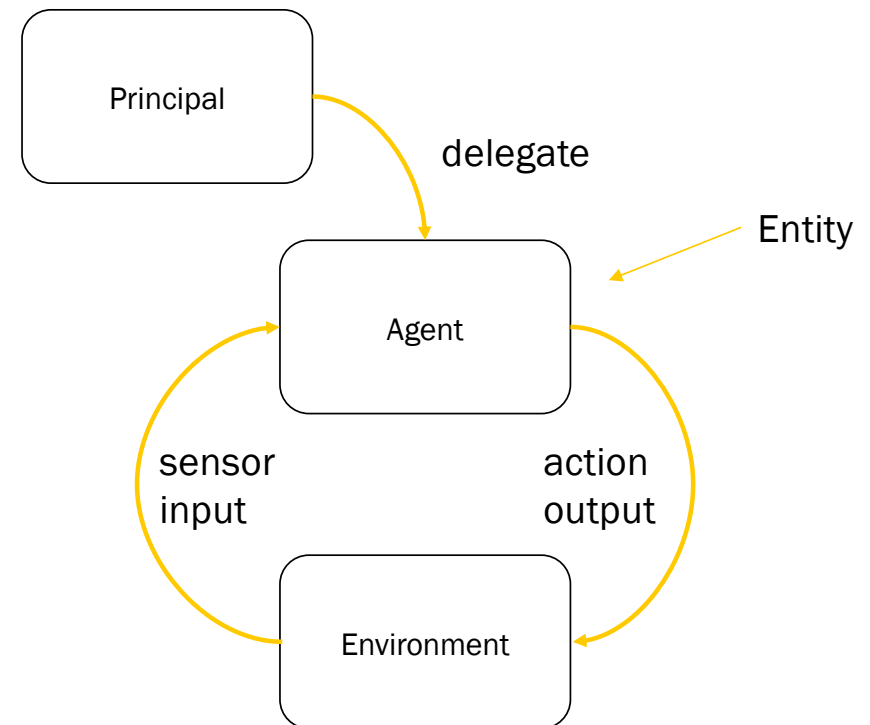
What is an Agent?

Oxford dictionary:

An agent is a person appointed by another person, known as the principal, to act on his or her behalf.

Wooldridge and Jennings (1995):

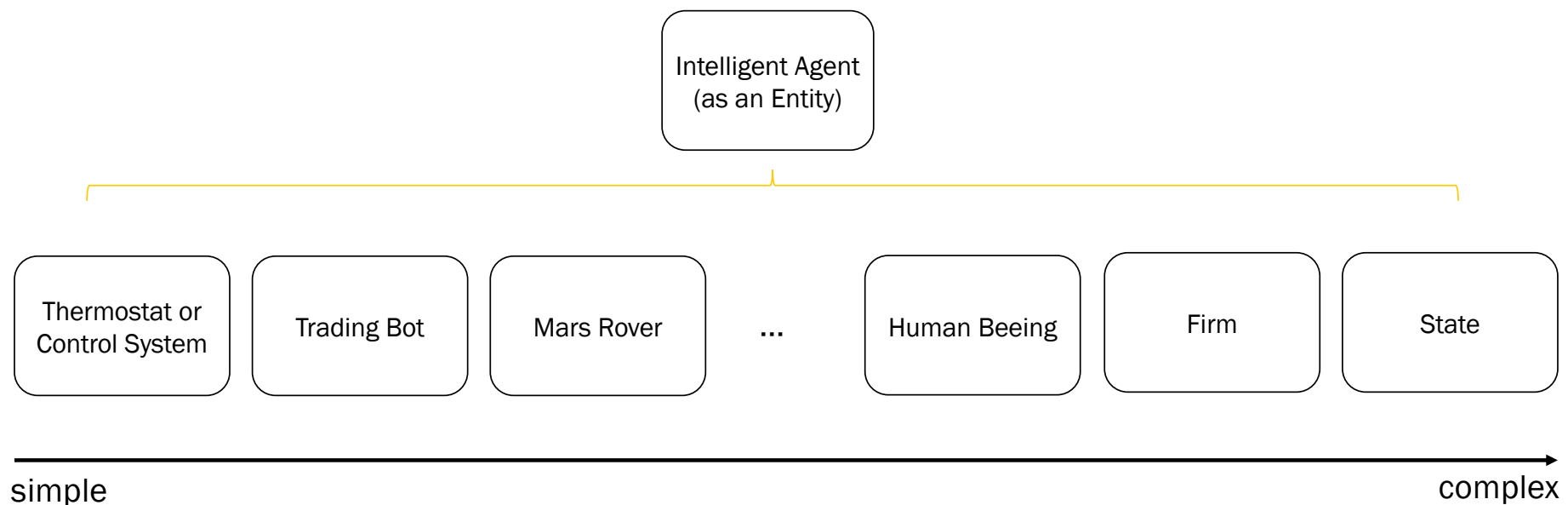
An agent is a computer system that is situated in some environment, and that is capable of autonomous action in this environment in order to meet its design objectives.



What is an Intelligent Agent?

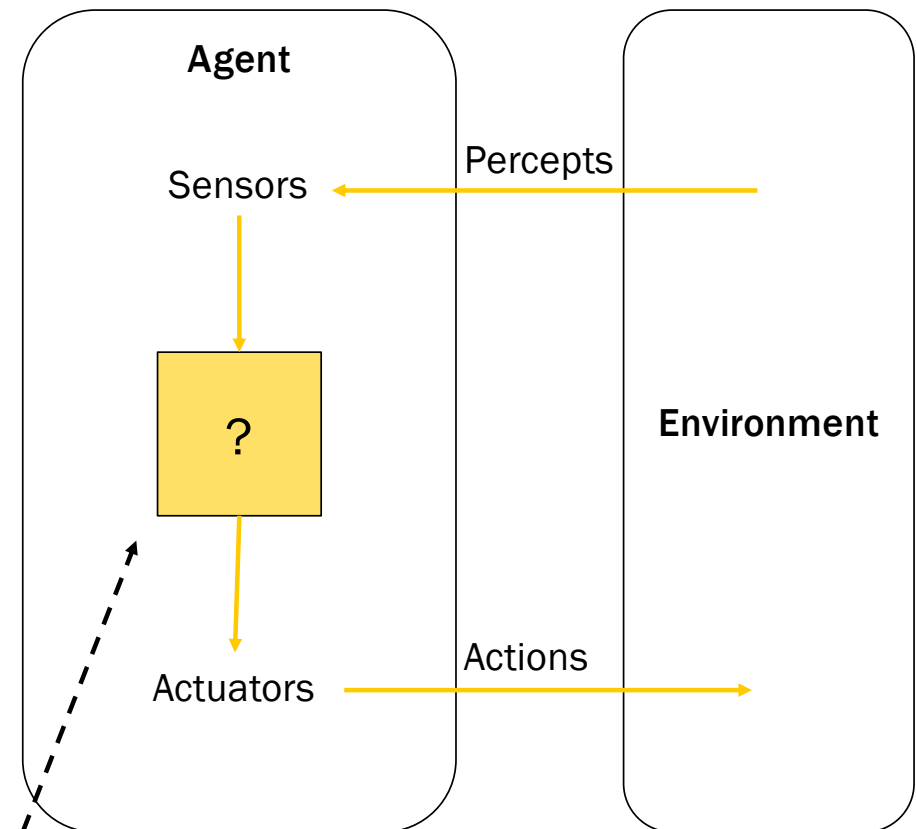
Our Definition: (from wikipedia):

An intelligent agent (IA) is an agent that perceives its environment, takes actions autonomously in order to achieve goals, and may improve its performance with learning or acquiring knowledge.



This leads to several questions

1. When does an agent act rationally?
2. What is the best way to describe the problem we want to solve with the agent?
3. How does the process of getting from sensor input to action output work?
4. What are the capabilities an agent should have?
5. How does the properties of the environment influence the design of an agent?
6. When does an agent should have the ability to learn?
7. What are the limitation of a (single) agent?



The **agent function** maps any given percept sequence to an action.

When does an agent act rationally?

Rationality at any given time depends on four things:

1. The performance measure that defines the criterion of success.
2. The agents prior knowledge of their environment (,its beliefs).
3. The actions that the agent can perform.
4. The agent's percept sequence to date.



When does an agent act rationally?

This leads to a definition of a rational agent

For each possible percept sequence, a rational agent should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever build-in knowledge the agent has.

(Russel & Norvig, Chapter 2.2.1)



What is the best way to describe the problem we want to solve with the agent?

Russel & Norvig call the problem we want to solve with an agent a **task environment**.

A task environment can be specified by the PEAS (Performance, Environment, Actuators, Sensors) description:

Agent Type	Performance Measure	Environment	Actuators	Sensors
Automated Taxi Driver	Safe, fast, legal, comfortable trip, maximize profits: $\max(\text{revenue} - \text{cost})$	Roads, other raffic, pedestrians, customer	Steering, accelerator, brake, signal, horn, display	Cameras, LIDAR, sonar, speedometer, GPS, odometer, accelerometer, engine sensors, keyboard
Trading Bot	Risk Level maximize profits: $\max(\text{selling} - \text{buying})$	Online Trading Platform & Social Network Platform	Selling and buying	knowledge retrieval: • scanning social network platforms • scanning trading platforms



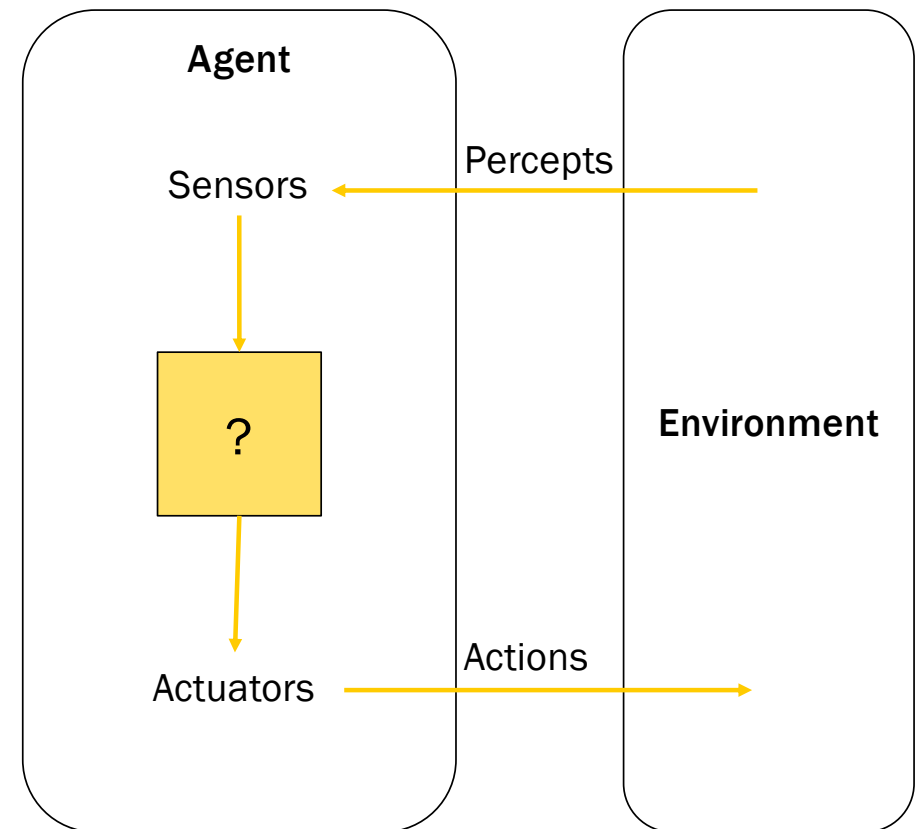
How does the process of getting from sensor input to action output work?

Russel & Norvig distinct between the agent function and the agent program:

The **agent function** that maps any given percept sequence to an action is a mathematical description:

$F: (p_1, p_2, \dots p_n) \rightarrow A$, where A is a possible action.

The **agent program** is the concrete implementation of the agent function.



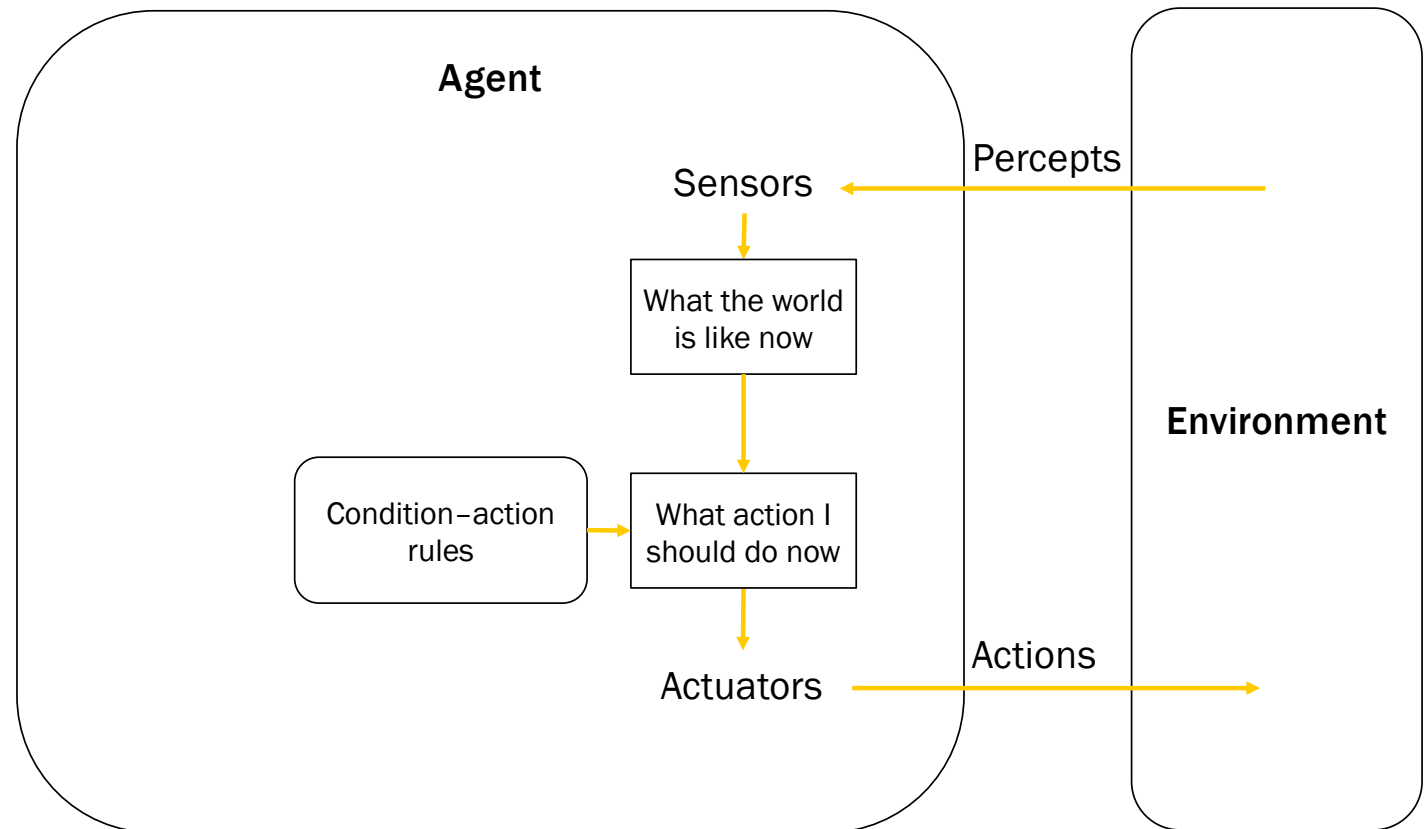
How does the process of getting from sensor input to action output work?

Russel & Norvig distinguish between different types of agent programs:

Agent Types	
Simple reflex agents	Selects action on current percept sequence using a condition-action rule (if... then ...). Simple but of limited intelligence and the environment should be fully observable.
Model-based reflex agents	Maintain internal state to keep track of the part of the world it can't see now. (Short-term memory). In addition he needs some model of the world to understand how actions would affect the world.
Goal-based agents (Utility based agents)	The agent has to find a way to achieve its goal. This can be done by reasoning, search and planning capabilities. Achieving a goal can be replaced by an utility function. Goal-based agents act to achieve their goals. Utility-based agents try to maximize their own expected „happiness“ (measured by an utility function).

How does the process of getting from sensor input to action output work?

1. Simple reflex agents:



How does the process of getting from sensor input to action output work?

2. Model-based reflex agents

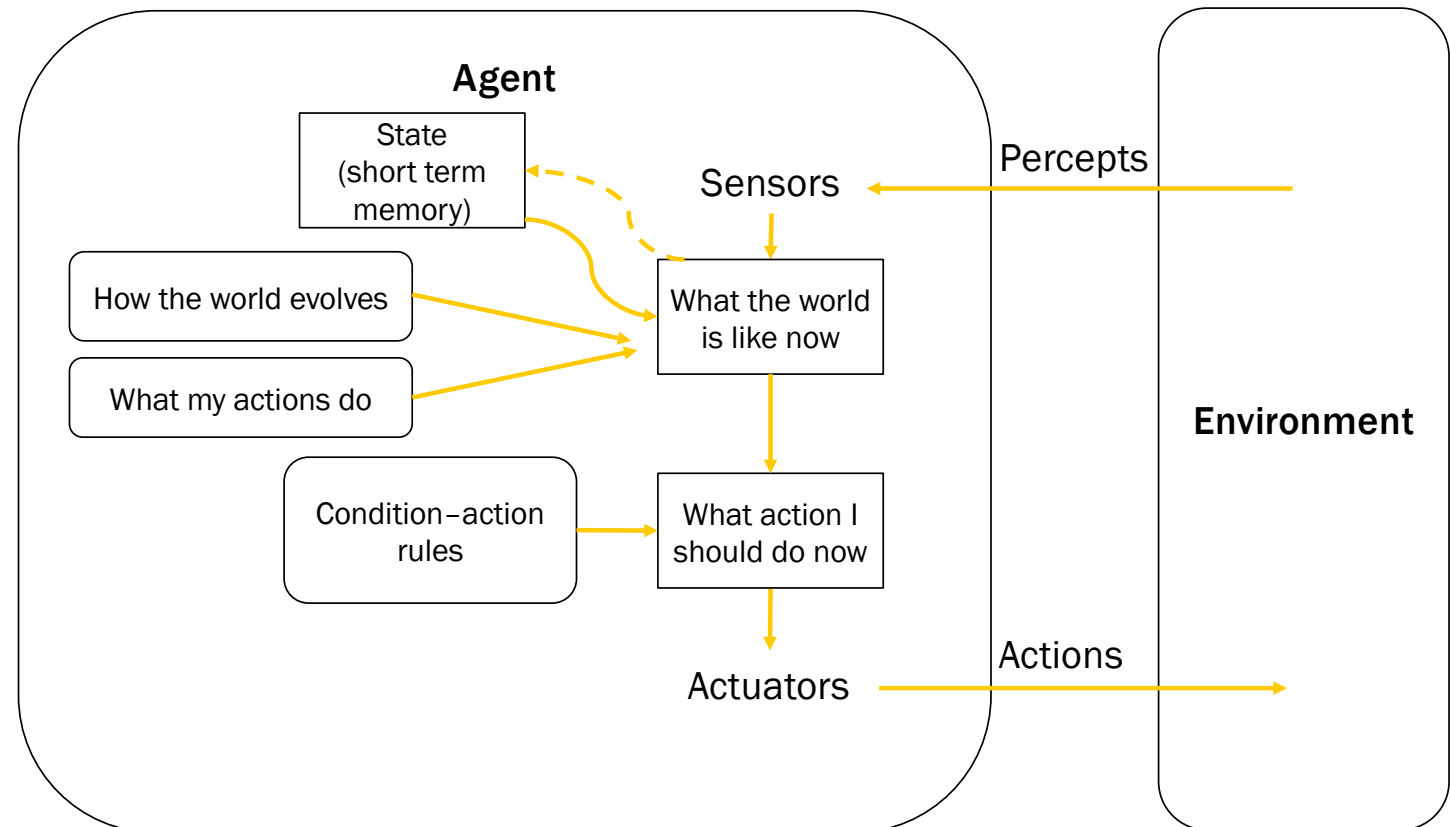
Agent can observe the environment only partially:

He needs

- a history and
- a model of the world

to complete the observation based on perception and world knowledge.

Next action is taken like simple reflex agents do.



How does the process of getting from sensor input to action output work?

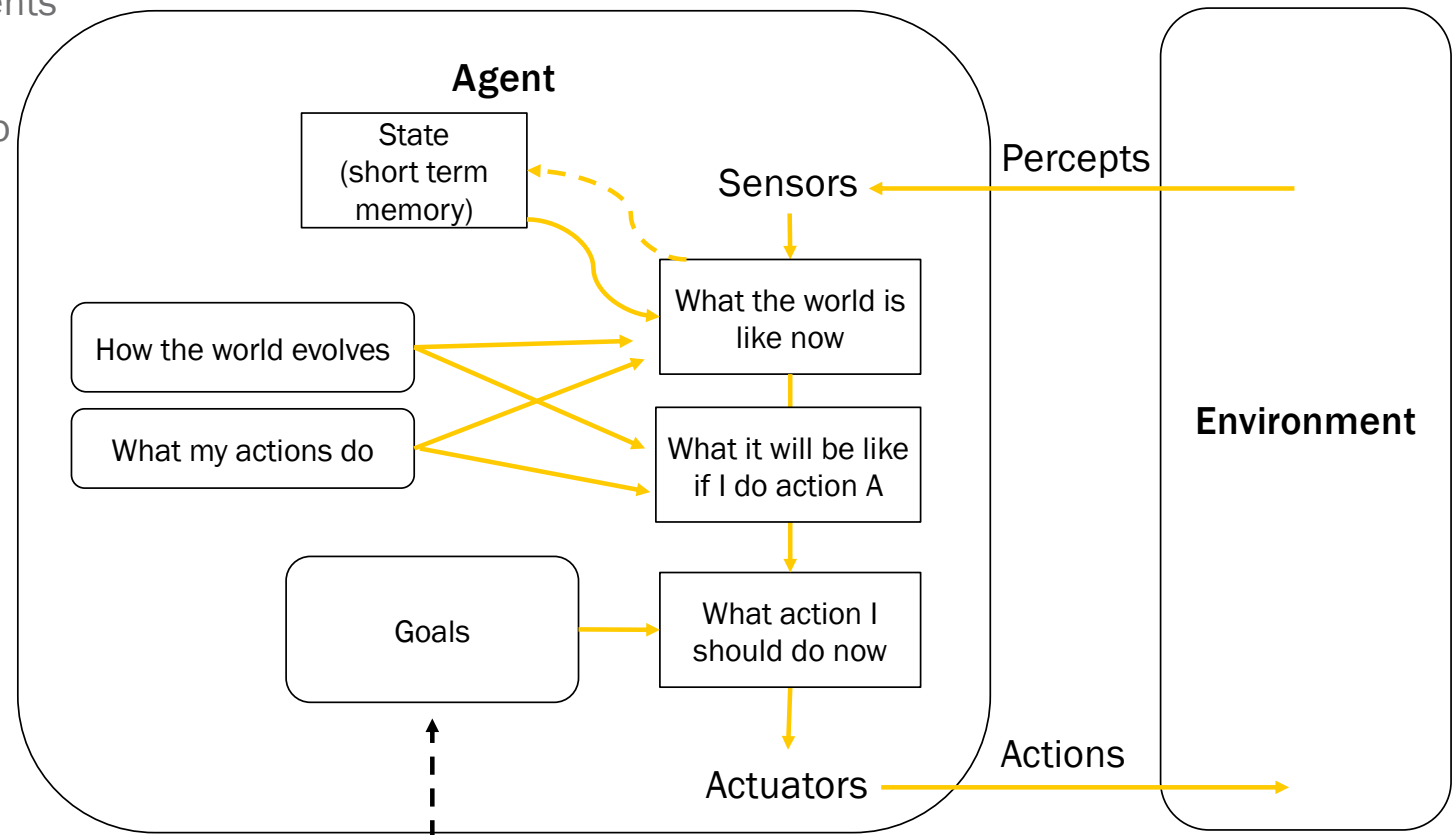
3. Goal- (& Utility-)based agents

The agent has to find a way to achieve its goal. This can be done by

- reasoning,
- search and
- planning capabilities

Decision making is fundamentally different from condition-action rules:

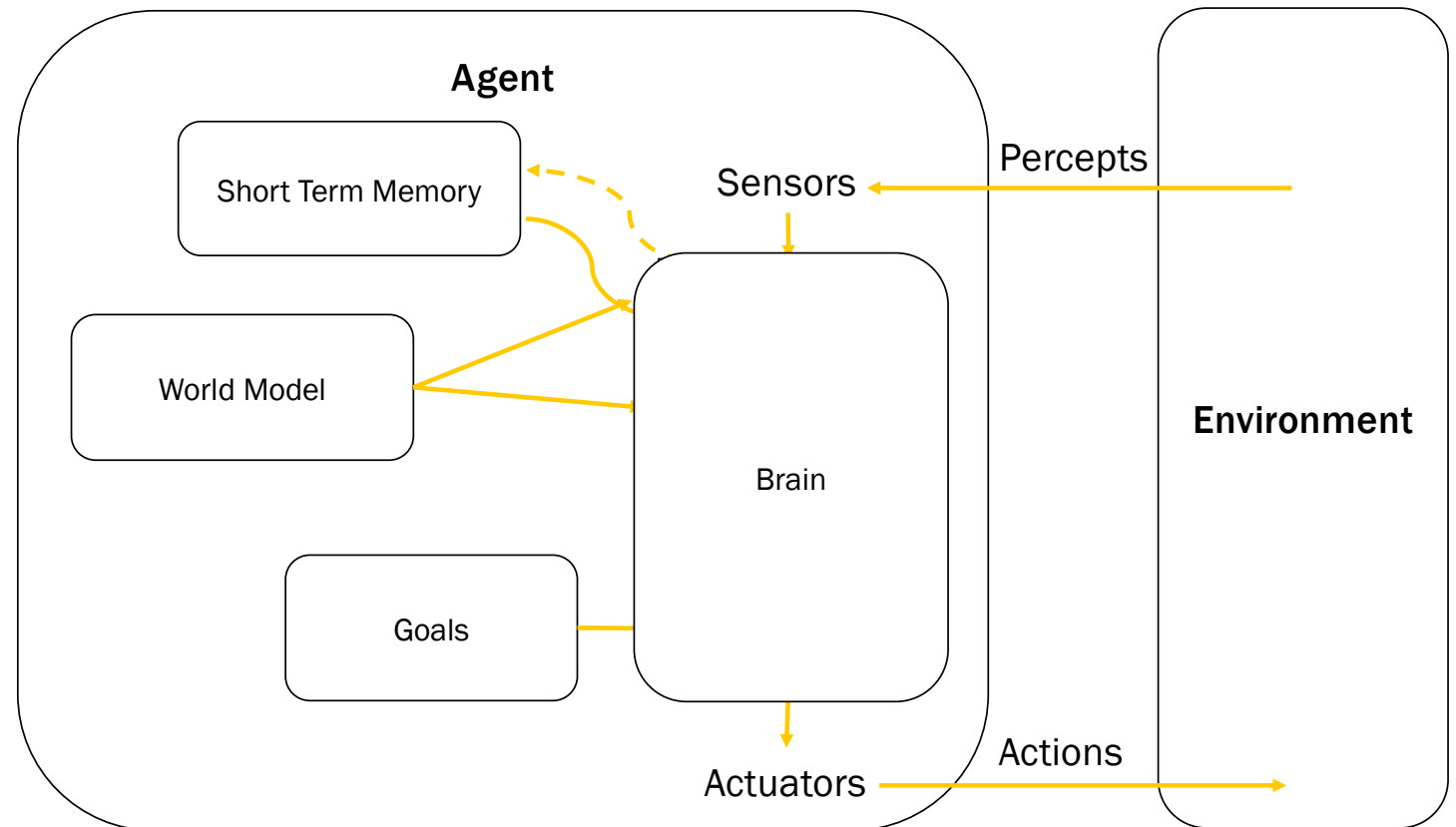
- What will happen if i do ...
- Will that make me happy



You could change the goal by an utility function.



Basic Structure of an intelligent Agent



What are the capabilities an agent should have (in addition to rationality)?

Property	
Autonomy	The agent can act without human intervention
Reactivity (Sensing & Acting)	The agent receives sensory input and it performs some action
Proactiveness	The agent exhibit goal-directed behavior by take the initiative
Social ability	The agent is friendly
Coordination	The agent is able to perform activities in a shared environment with other agents.
Cooperation	The agent is able to coordinate with other agents to achieve a common purpose.
Learning & adaptability	The agent is able to i) adopt to changing environments, ii) take goal-directed initiatives, and iii) learn from its own experiences
Personality	An agent has a well-defined (believable) personality (and emotional state)
Resource limitation	The agent can only act as long it has resources.
Communication ability	The agent is able to communicate with other agents with language (human-like)
Prediction & Planning	The agent is able to do planning
Reasoning	The agent is able to reason

02 Reasoning



„Cogito, Ergo sum“

(René Descarts)

It is thinking, not language, that lies closest to the core of our individual identity.

What is Thining?

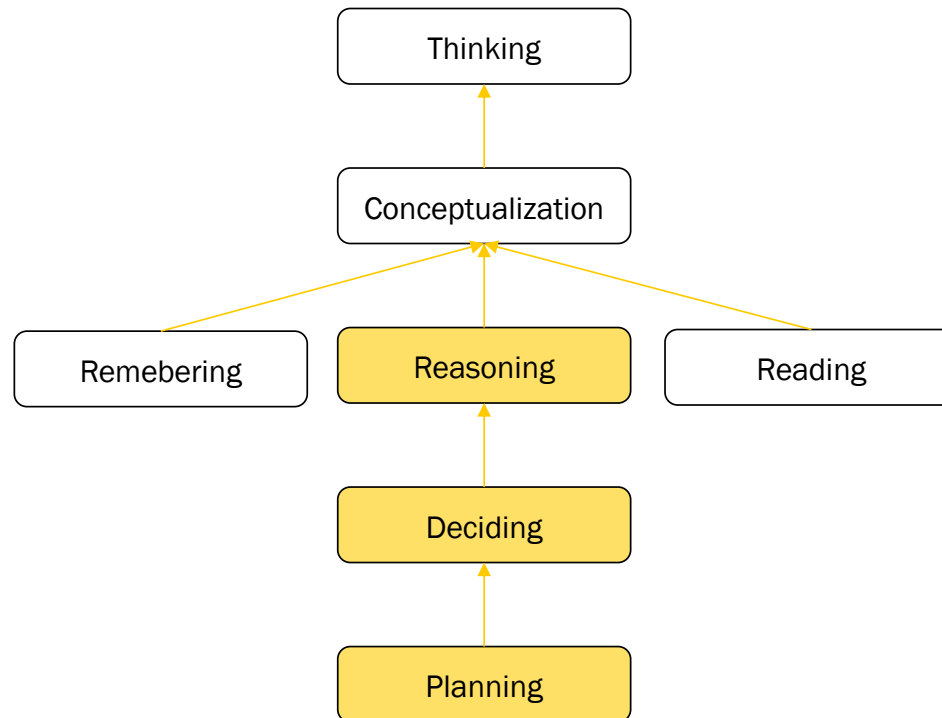
Definition from „The Cambridge Handbook of Thinking and Reasoning“:

- Thinking is the systematic transformation of mental representations of knowledge to characterize actual or possible states of the world, often in service of goals.
- A mental representation of knowledge is an internal description that can be manipulated to form other descriptions. To count as thinking, the manipulations must be systematic transformations governed by certain constraints.
- Thinking often seems to be a conscious activity of which the thinker is aware (cogito, ergo sum).



What is Thining?

Mental activities:



What is Reasoning?

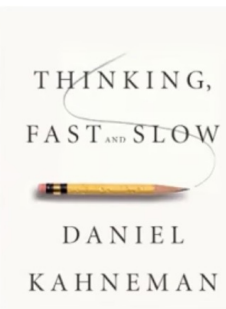
Definition from „The Cambridge Handbook of Thinking and Reasoning“:

- **Reasoning** places emphasis on the process of drawing inference (conclusions) from some initial information (premises).
- An inference is **deductive** if the truth of the premises guarantees the truth of the conclusion.
- If the truth of the premises renders the truths of the conclusion more credible but does not bestow certainty, the inference is called **inductive**.
- **Problem solving** involves the construction of a course of action that can achieve a goal.

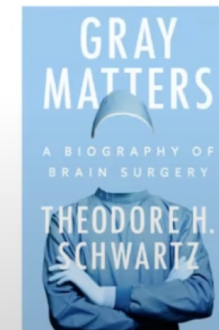


Reasoning in Humans

- There different types of reasoning that humans can follow:
 - Deductive reasoning: drawing conclusions from a set of axioms or premises, as seen in mathematics.
 - Inductive reasoning: generalizing from specific examples to make broader statements.
 - Probabilistic reasoning: making likely predictions based on observed data, with the probability of being correct increasing with more evidence.
 - Analogical reasoning: comparing a situation to a similar one and extrapolating conclusions.
 - Abductive reasoning: finding the most likely explanation for an event or observation.
- These types of reasoning are also compared to human cognitive processes:
 - System 1 thinking (automatic decision-making) is similar to zero-shot reasoning in LLMs.
 - System 2 thinking (deliberate and reflective reasoning) aligns with more intentional forms of reasoning, such as deductive and abductive reasoning.



"Dr Schwartz believes that most of these feelings arise spontaneously or in response to triggers from the outside world. The brain then generates a plausible rationale post hoc for why a movement was made or a thought conjured."
Source: The Economist



Is a Large Language Model able to do reasoning?

Is a Large Language Model capable of drawing conclusions?

Can Large Language Model adopt to novelty?

There is always some sense of reasoning!

Two ways the term reasoning is used:

Memorizing and applying a program:

- Memorizing an algorithm and applying it to an unseen situation? (Learning an Algo for multiplying numbers at school and applying it in a test).

Recombine to adopt to novelty:

- Faced with something not seen before and you have to recompose or recombine cognitive building blocks from your knowledge to solve the problem.



02 B Large Reasoning Models



Artificial General Intelligence (AGI) by Francois Chollet

On the Measure of Intelligence

François Chollet *
Google, Inc.
fchollet@google.com

November 5, 2019

Abstract

To make deliberate progress towards more intelligent and more human-like artificial systems, we need to be following an appropriate feedback signal: we need to be able to define and evaluate intelligence in a way that enables comparisons between two systems, as well as comparisons with humans. Over the past hundred years, there has been an abundance of attempts to define and measure intelligence, across both the fields of psychology and AI. We summarize and critically assess these definitions and evaluation approaches, while making apparent the two historical conceptions of intelligence that have implicitly guided them. We note that in practice, the contemporary AI community still gravitates towards benchmarking intelligence by comparing the *skill* exhibited by AIs and humans at specific tasks, such as board games and video games. We argue that solely measuring skill at any given task falls short of measuring intelligence, because skill is heavily modulated by prior knowledge and experience: unlimited priors or unlimited training data allow experimenters to “buy” arbitrary levels of skills for a system, in a way that masks the system’s own generalization power. We then articulate a new formal definition of intelligence based on Algorithmic Information Theory, describing intelligence as *skill-acquisition efficiency* and highlighting the concepts of *scope*, *generalization difficulty*, *priors*, and *experience*, as critical pieces to be accounted for in characterizing intelligent systems. Using this definition, we propose a set of guidelines for what a general AI benchmark should look like. Finally, we present a new benchmark closely following these guidelines, the Abstraction and Reasoning Corpus (ARC), built upon an explicit set of priors designed to be as close as possible to innate human priors. We argue that ARC can be used to measure a human-like form of general fluid intelligence and that it enables fair general intelligence comparisons between AI systems and humans.

The consensus definition of AGI,

"a system that can automate the majority of economically valuable work,"

while a useful goal, is an incorrect measure of intelligence.

- Measuring task-specific skill is not a good proxy for intelligence.
- Skill is heavily influenced by prior knowledge and experience. Unlimited priors or unlimited training data allows developers to “buy” levels of skill for a system. This masks a system’s own generalization power.
- Intelligence lies in broad or general-purpose abilities; it is marked by skill-acquisition and generalization, rather than skill itself.

<https://arcprize.org>



Artificial General Intelligence (AGI) by Francois Chollet

Here's a better definition for AGI:

„AGI is a system that can efficiently acquire new skills outside of its training data.“

More formally:

„The intelligence of a system is a measure of its skill-acquisition efficiency over a scope of tasks, with respect to priors, experience, and generalization difficulty.“

This means that a system is able to adapt to new problems it has not seen before and that its creators (developers) did not anticipate.

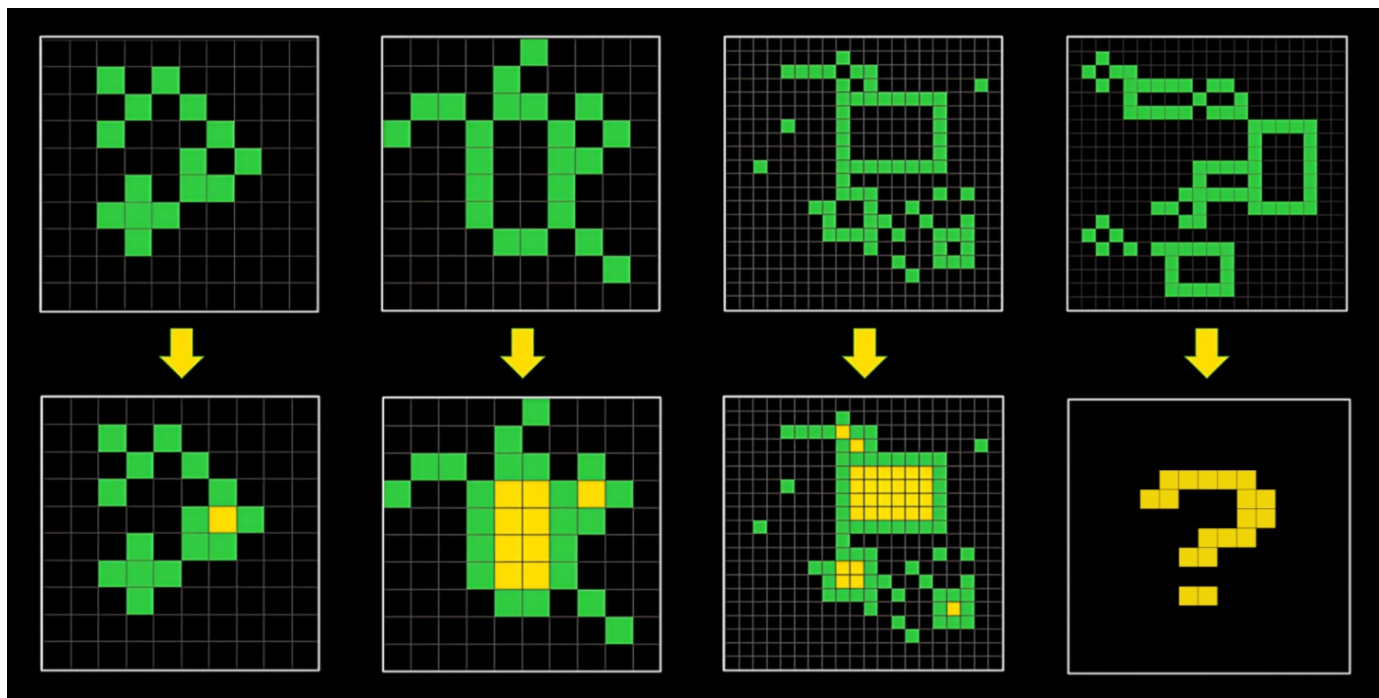
ARC-AGI is the only AI benchmark that measures our progress towards general intelligence.

<https://arcprize.org>



ARC-AGI Benchmark

ARC-AGI consists of unique training and evaluation tasks. Each task contains input-output examples. The puzzle-like inputs and outputs present a grid where each square can be one of ten colors. A grid can be any height or width between 1x1 and 30x30.



<https://arcprize.org>

ARC-AGI assumed Knowledge

ARC-AGI is explicitly designed to compare artificial intelligence with human intelligence. To do this, ARC-AGI explicitly lists prior knowledge human have to provide a fair ground for comparison. These "core knowledge priors" are ones that humans naturally possess, even in childhood.

Types of core Knowledge	
1. Objectness	Objects persist and cannot appear or disappear without reason. Objects can interact or not depending on the circumstances.
2. Goal-directedness	Objects can be animate or inanimate. Some objects are "agents" - they have intentions and they pursue goals.
3. Numbers & counting	Objects can be counted or sorted by their shape, appearance, or movement using basic mathematics like addition, subtraction, and comparison.
4. Basic geometry & topology	Objects can be shapes like rectangles, triangles, and circles which can be mirrored, rotated, translated, deformed, combined, repeated, etc. Differences in distances can be detected.

ARC-AGI avoids a reliance on any information that isn't part of these priors, for example acquired or cultural knowledge, like language.

<https://arcprize.org>



Impact

Solving ARC-AGI represents a material stepping stone toward AGI.

- At minimum, solving ARC-AGI would result in a new programming paradigm. It would allow anyone, even those without programming knowledge, to create programs simply by providing a few input-output examples of what they want.
- This would dramatically expand who is able to leverage software and automation. Programs could automatically refine themselves when exposed to new data, similar to how humans learn.

If found, a solution to ARC-AGI would be more impactful than the discovery of the Transformer. The solution would open up a new branch of technology.

<https://arcprize.org>



ARC-AGI High Scores

2024 ARC-AGI-PUB HIGH SCORES				
o3 (coming soon)	75.7%	62.6%		
Jeremy Berman	53.6%	56.5%	CODE	PAPER
MARA(BARC) + MIT	47.5%	62.6%	CODE	PAPER
Ryan Greenblatt	43%	42%	CODE	PAPER
o1-preview	16%	21%	CODE	
Claude 3.5 Sonnet	14%	21%	CODE	
GPT-4o	5%	9%	CODE	
Gemini 1.5	4.5%	6%	CODE	
Score (semi-private eval) / Score (public eval)				

<https://arcprize.org>



The Path to Reasoning Models

- Chain of Thoughts
- CoT with Self-Consistency
- Tree of Thoughts: Deliberate Problem Solving
- Test-Time Reasoning with Tree-Search Algorithms



CoT with Self-Consistency

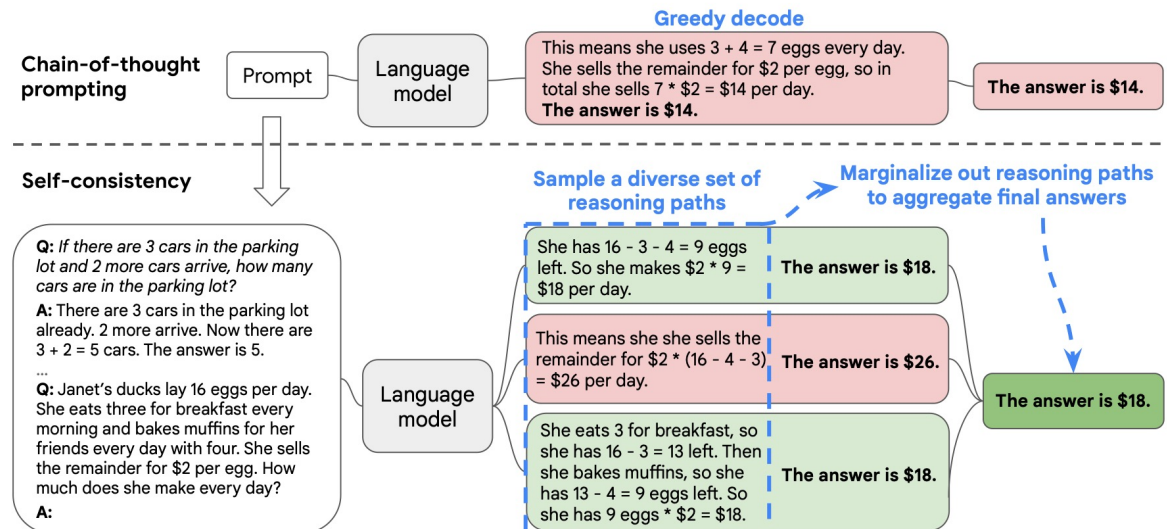
07.03.2023

SELF-CONSISTENCY IMPROVES CHAIN OF
THOUGHT REASONING IN LANGUAGE MODELS

<https://arxiv.org/pdf/2203.11171>

The self-consistency method contains three steps:

- (1) prompt a language model using chain-of-thought (CoT) prompting;
- (2) replace the “greedy decode” in CoT prompting by sampling from the language model’s decoder to generate a diverse set of reasoning paths; and
- (3) marginalize out the reasoning paths and aggregate by choosing the most consistent answer in the final answer set.



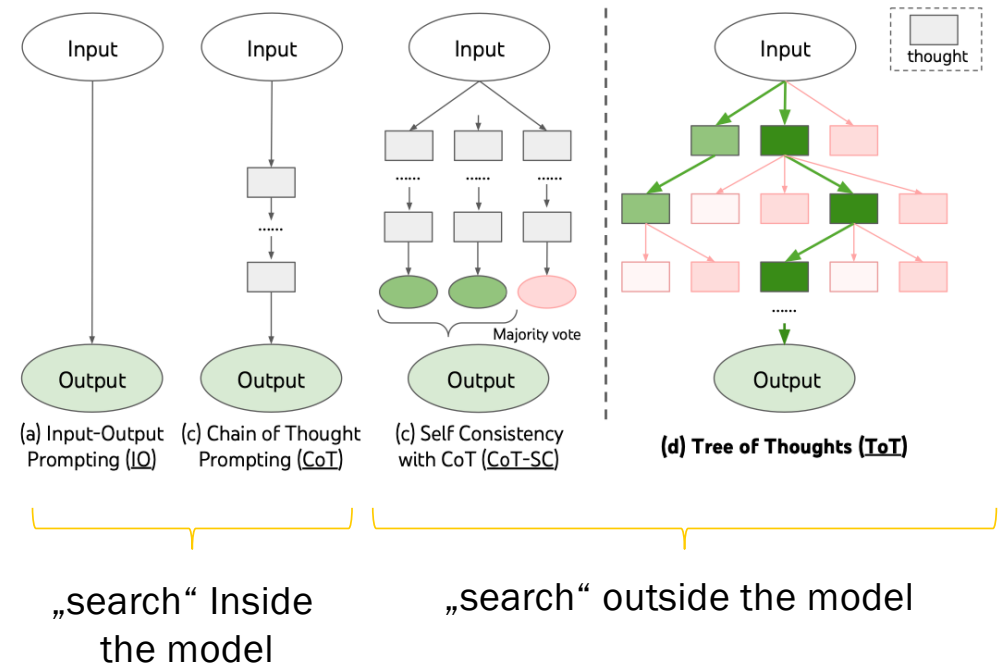
Tree of Thoughts - a new framework for language model inference

Language models are increasingly being deployed for general problem solving across a wide range of tasks, but are still confined to token-level, left-to-right decision-making processes during inference.

- "Tree of Thoughts" (ToT) generalizes over the popular "Chain of Thought" approach and enables exploration over coherent units of text ("thoughts") that serve as intermediate steps toward problem solving.
- ToT allows LMs to perform deliberate decision making by considering multiple different reasoning paths and self-evaluating choices to decide the next course of action, as well as looking ahead or backtracking when necessary to make global choices.
- Code repo with all prompts: <https://github.com/princeton-nlp/tree-of-thought-llm>

Dez 2023

<https://arxiv.org/pdf/2305.10601>



Test-Time Reasoning with Tree-Search Algorithms

A novel method for test-time reasoning enhancement, combining search- based algorithms and reward-based feedback to solve complex tasks.

Address limitations in existing approaches, such as chain-of-thought reasoning and self-consistency, BY

1. Balance exploration and exploitation in the solution space, enabling structured and informed decision-making.
2. Allow reasoning steps to be dynamically evaluated and improved during inference.

Paper:

- Technical Report: Enhancing LLM Reasoning with Reward-guided Tree Search, <https://arxiv.org/pdf/2411.11694>

A reward-guided tree search framework comprising:

1. policy model trained to align with reasoning fermats.
2. reward model providing outcome-level or process-level feedback.
3. Monte Carlo Tree Search (MCTS) algorithm to explore broader solution spaces efficiently.



03 Intelligent Agents with LLM as their Brain



Vertical Intelligence Adoption (The Rise of Vertical AI Agent Solutions)

Vertical AI agents

Vertical AI agents have emerged as a transformative solution, embedding industry-specific expertise and fine-tuned intelligence into adaptable, realtime systems:

1. Targeted Domain Expertise: tailored for specific industries, utilizing domain-specific reasoning engines (LLMs) fine-tuned for specialized knowledge and workflows to address complex challenges effectively
2. Dynamic Adaptability in Real-time Operations: excel in dynamic environments, continuously adapting to changing conditions and operational demands.
3. End-to-End Workflow Automation: transform raw data into actionable outcomes, streamlining workflows traditionally reliant on human intervention.

Agentic Systems: A Guide to Transforming Industries with Vertical AI Agents

Fouad Bousetouane^{1,2}

¹The University of Chicago, USA

²2nd sight.ai

bousetouane@uchicago.edu

Abstract

The evolution of agentic systems represents a significant milestone in artificial intelligence and modern software systems, driven by the demand for vertical intelligence tailored to diverse industries. These systems enhance business outcomes through adaptability, learning, and interaction with dynamic environments. At the forefront of this revolution are Large Language Model (LLM) agents, which serve as the cognitive backbone of these intelligent systems.

In response to the need for consistency and scalability, this work attempts to define a level of standardization for Vertical AI agent design patterns by identifying core building blocks and proposing a **Cognitive Skills** Module, which incorporates domain-specific, purpose-built inference capabilities. Building on these foundational concepts, this paper offers a comprehensive introduction to agentic systems, detailing their core components, operational patterns, and implementation strategies. It further explores practical use cases and examples across various industries, highlighting the transformative potential of LLM agents in driving industry-specific applications.

arXiv:2501.00881v1 [cs.MA] 1 Jan 2025

<https://arxiv.org/abs/2501.00881>



Vertical Intelligence Adoption (The Rise of Vertical AI Agent Solutions)

General-Purpose Systems:

The traditional IT- systems and platforms that serve as the backbone of business operations, offer reliable tools for managing workflows and maintaining operational consistency. Their architecture emphasizes horizontal scalability and general applicability, enabling businesses to standardize processes and optimize routine tasks across industries.

These general-purpose systems are made for managing repetitive tasks and scaling operations with rule-based automation and structured data inputs.

While effective for predictable and routine processes, they fall short in addressing domain-specific tasks that require contextual intelligence and adaptability.

Context-Aware Systems:

Context-aware systems aim to address these gaps by integrating real-time data and adaptability into workflows. Context-aware systems are designed to:

- Understand dynamic environments: Incorporate real-time data to adjust workflows and outputs.
- Bridge data to decisions: Translate raw data into actionable insights without extensive manual intervention.
- Adapt to evolving scenarios: Adjust to unforeseen conditions or emerging trends.

While they offer adaptability, they are constrained by their dependence on predefined rules and lack the ability to process unstructured data or make advanced contextual decisions.

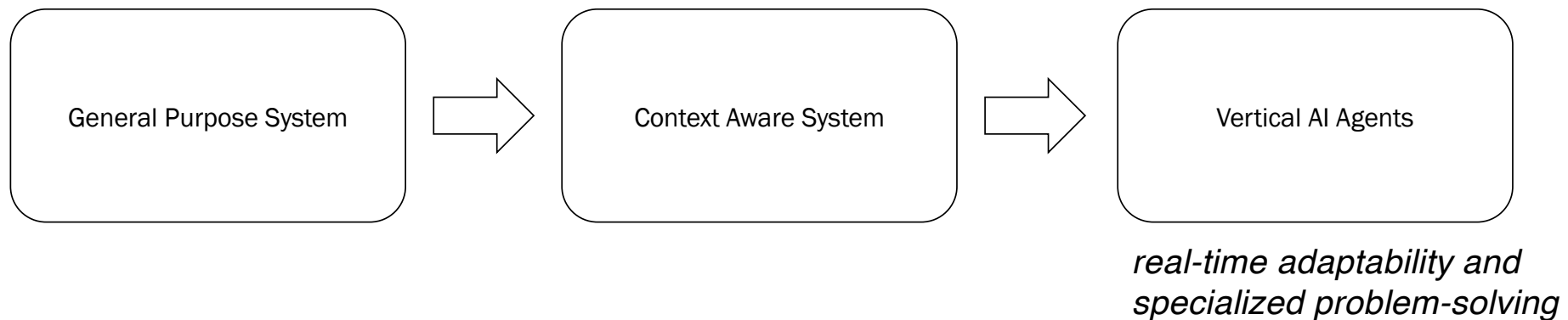


Vertical Intelligence Adoption

Vertical AI agents:

Vertical AI agents have emerged as a transformative solution, embedding industry-specific expertise and fine-tuned intelligence into adaptable, realtime systems. By combining the flexibility of context-aware systems with domain knowledge, they empower organizations to address unique challenges with precision and efficiency.

Evolution directed to intelligent system design:



LLM Agents

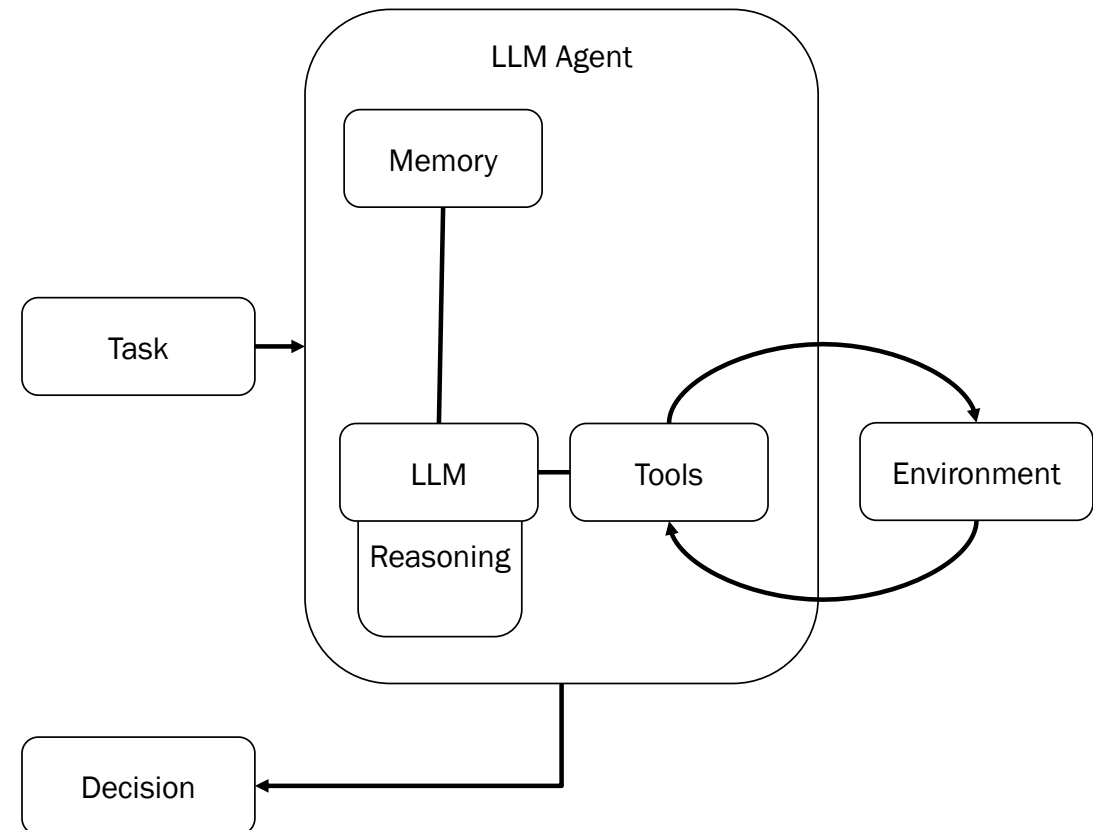
LLM Agents:

LLM agents are autonomous, intelligent systems powered by Large Language Models (LLMs) that integrate modular components (reasoning, memory, cognitive skills, and tools) to solve complex tasks in dynamic and evolving environments.

These agents are designed to operate independently, adapt to changes, and execute sophisticated tasks by combining domain-specific expertise with contextual understanding.

Modular Architecture: Each module within the agent's architecture serves a distinct purpose:

- reasoning enables logical decisionmaking,
- memory supports retention and recall of critical information, and
- tools facilitate interaction with external systems and environments.



<https://arxiv.org/abs/2501.00881>



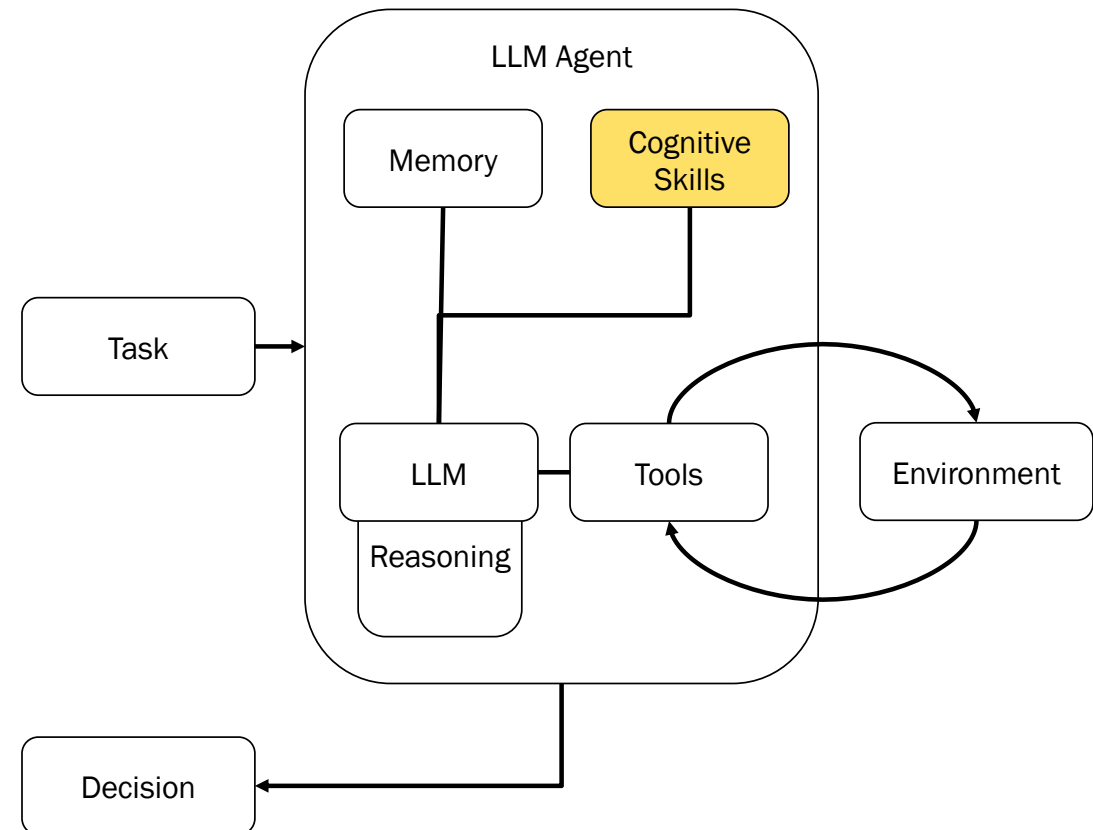
LLM Agents

LLM Agents:

Introducing a new component: Cognitive Skills fills the gap between

- pre-trained or finetuned LLM reasoning,
- external tools for interacting with the environment,
- and new inference models.

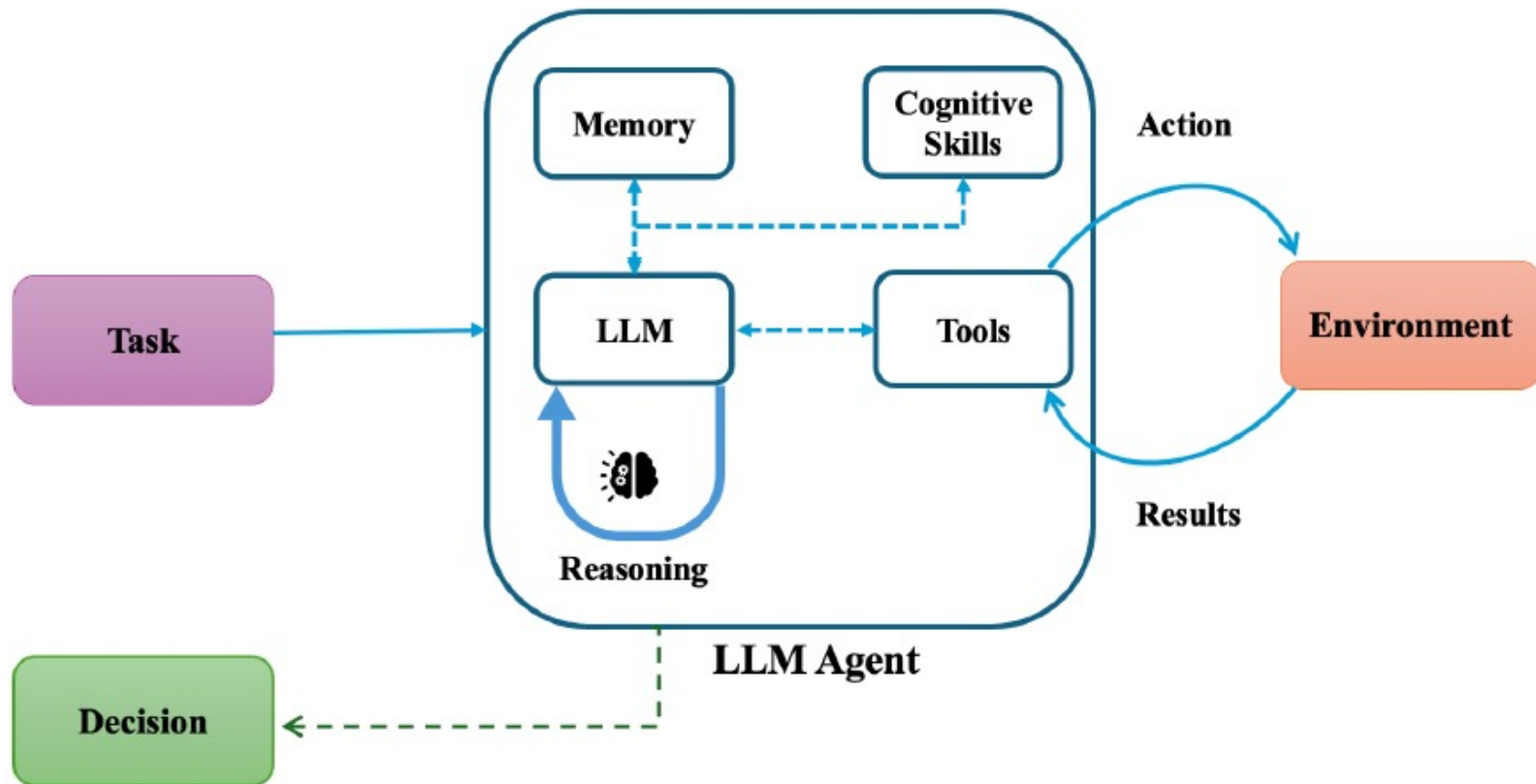
This module ensures that LLM agents are equipped with purpose-built models tailored to specific tasks, enhancing their ability to operate effectively across various domains and challenges.



<https://arxiv.org/abs/2501.00881>



Architecture of Core Components of an LLM Agent (Backup)



<https://arxiv.org/abs/2501.00881>, Figure 1

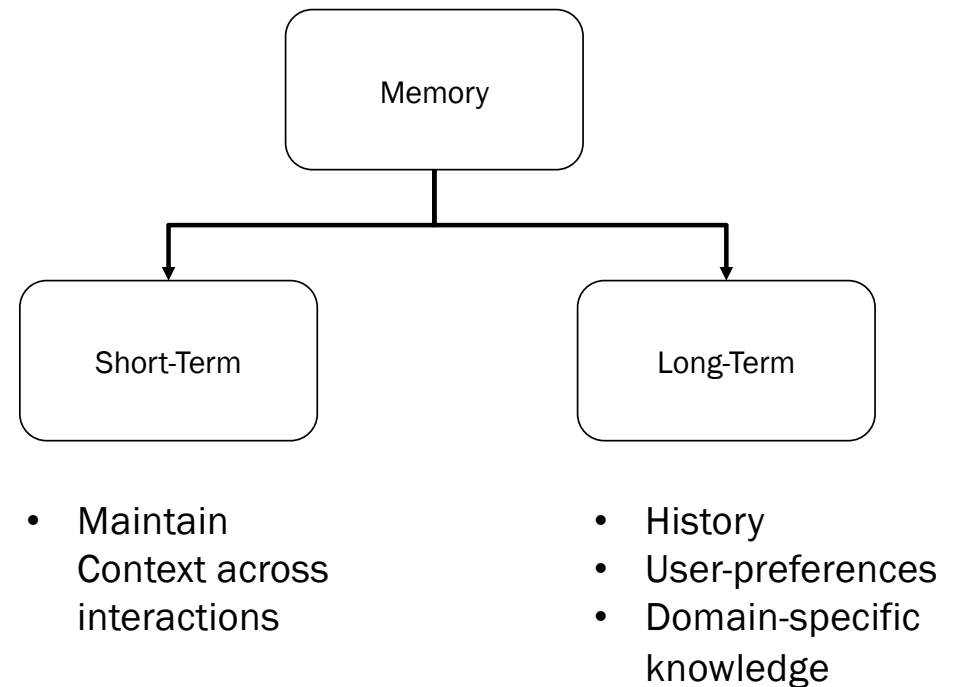
Core Modules of LLM Agents

Memory:

Is the agent's ability to maintain context across interactions, ensuring personalized and consistent responses.

It stores historical interactions, user preferences, and domain-specific knowledge, serving as the agent's long-term storage system.

- Contextual Awareness: Drawing on prior interactions to maintain continuity.
- Personalization: Adapting responses based on user-specific information.
- Domain Expertise: Utilizing stored knowledge to deliver precise and informed outputs.



Core Modules of LLM Agents

LLM as Reasoning-Engine:

Acts as the brain of the LLM agent.

Is the decision-making core of an LLM agent. It orchestrates logical inference, planning, contextual understanding, and personalized interaction, transforming raw data into actionable insights.

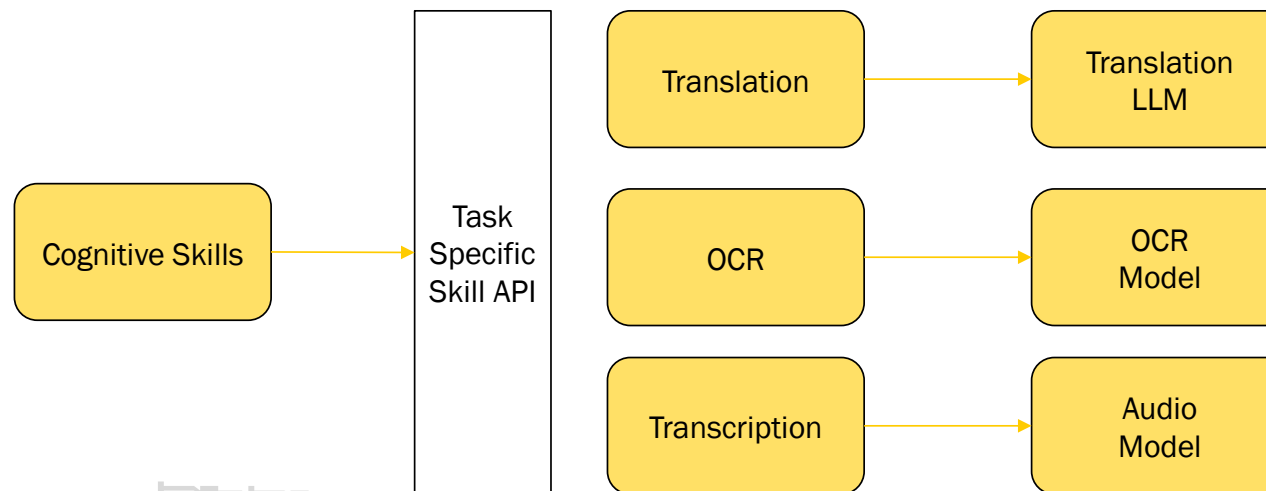
- Logical Inference and Problem-Solving
- Contextual Understanding and Response Generation
- Task Sequencing, Goal-Oriented Planning, and Chain of Thought Reasoning
- Adaptive Personas for Tailored Interactions



Core Modules of LLM Agents

Cognitive Skills:

- enable Task-specific inference.
- acts as a model hub, equipping the agent with purpose-built models specifically designed to accomplish tasks that general purpose LLMs, even when fine-tuned LLMs, struggle to perform effectively.
- This could be implemented by providing them as services

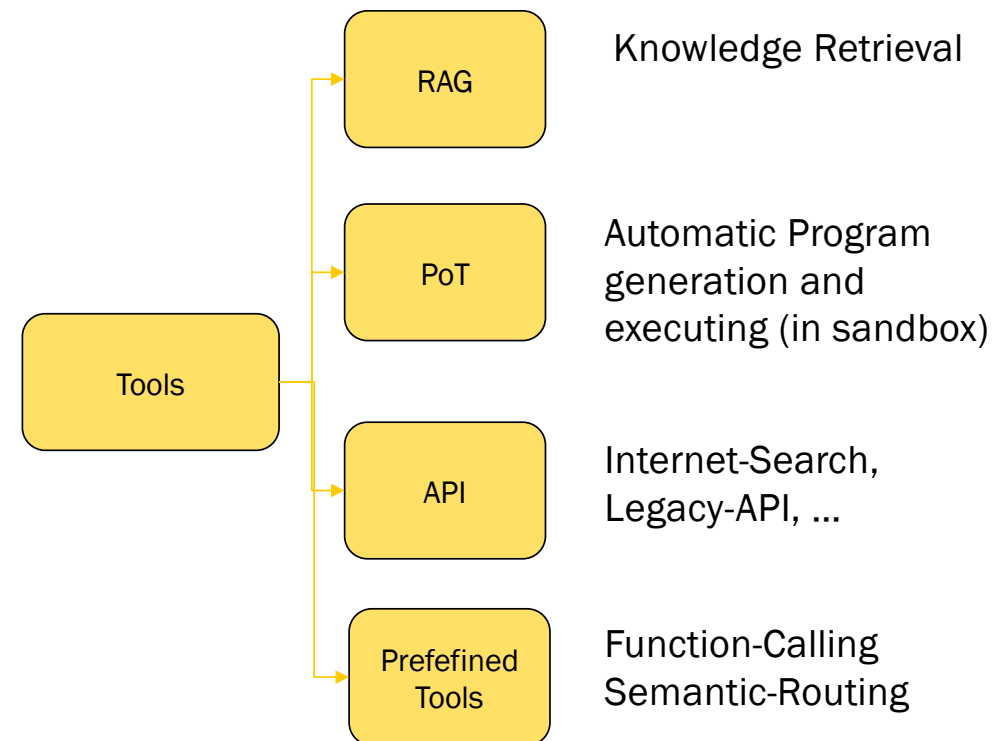


Core Modules of LLM Agents

Tools:

Tools are bridging knowledge and interaction.

- Knowledge Retrieval Systems: Retrieval-Augmented Generation (RAG) systems to access structured (e.g., databases) and unstructured (e.g., document repositories) knowledge, enabling the agent to incorporate relevant domain-specific information into its operations.
- Dynamic API Integration: Tools that allow the agent to interact with live data streams, proprietary platforms, and external systems, facilitating real-time decision-making and adaptive responses.
- Legacy System Interfaces: Tools for bridging traditional structured data systems, such as relational databases, to incorporate historical data and insights into the agent's current tasks.
- Contextual Awareness Tools: Systems that provide the agent with situational and environmental context, enabling it to tailor its actions and outputs based on specific operational scenarios.



03 B Ingredients



Start with simple Things

LLM augmented Agents

- Finetune LLM for being an agent
- Pydantic and Instructor.py: force llm to structured output
- Function Calling
- Program of Thoughts
- PydanticAI: as basic Agent Framework





Pydantic - <https://docs.pydantic.dev/latest/>

Pydantic is the most widely used data validation library for Python. It uses Python type hints to define data models and automatically validates, parses, and enforces types at runtime, ensuring data consistency.

Pydantic is commonly used in frameworks that work with large language models (LLMs). Its data validation and serialization capabilities make it a great fit for managing structured input and output data in LLM-related workflows.

The output from an LLM can be mapped to a Pydantic model to validate and process the results, ensuring they meet the expected structure.

Examples in LLM Frameworks

- FastAPI Integration: Many LLM-based APIs are built using FastAPI, which uses Pydantic for request and response models.
- LangChain: LangChain, a popular framework for chaining LLM prompts and tools, utilizes Pydantic to define and validate schemas for tools, chains, and agent actions.

Its declarative style and tight integration with Python's type system make Pydantic a natural choice for robust LLM frameworks.



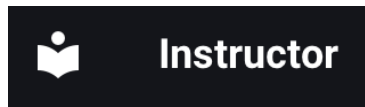
Some Pydantic Example Code

```
from pydantic import BaseModel, ValidationError
# Define a Pydantic model
class User(BaseModel):
    id: int
    name: str
    email: str
    age: int

try:
    valid_user = User(id=1, name="Alice", email="alice@example.com", age=25)
    print("valid user:", valid_user.dict()) # Convert model to dictionary
    invalid_user = User(id=1, name="Alice", email="alice@example.com", age="twenty five")

except ValidationError as e:
    print(e.json()) # Handle validation errors
```





Instructor – <https://python.useinstructor.com>

Instructor is the Most Popular Library for Simple Structured Outputs. Instructor makes it easy to get structured data like JSON from LLMs.

Instructor sits on top of Pydantic.

The basic idea is to force the output of the LLM into a particular standard so that it can be reused programmatically.



Function Calling

```
messages = [  
  {  
    'role': 'system',  
    'content': 'You are a helpful customer support assistant. Use the supplied tools to assist the user."',  
  },  
  {  
    'role': 'user',  
    'content': 'What is the customer id of the customer with email adress dominik.marc.neumann@icloud.com',  
  },  
]
```



Function Calling

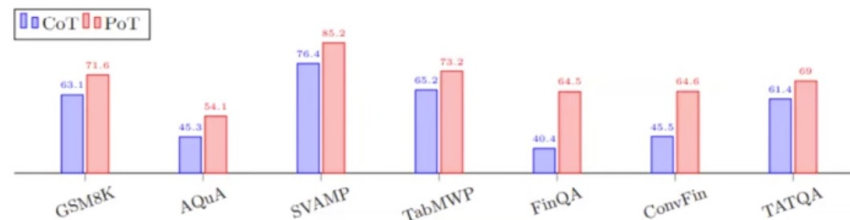
```
tools = [{
    "type": "function",
    "function": {
        "name": "get_customer_id",
        "description": "Get the customer id for a customer with a given email adress. Call this whenever you need
                        to know the customer id, for example when a somebody asks 'Get the customer id for the
                        following email address'",
        "parameters": {
            "type": "object",
            "properties": {
                "email_adress": {
                    "type": "string",
                    "description": "The email adress of the customer."}
            },
            "required": ["email_adress"],
            "additionalProperties": False }}}]

response = client.chat.completions.create(
    model="llama3.2:latest",
    messages=messages,
    tools=tools,
)
```



Program of Thoughts

- "Program of Thoughts" (PoT) (W. Chen et al. 2023) involves using LLMs to generate programs that express complex reasoning procedures, and then executing these programs using an **external (Python) interpreter**.
- The methodology combines LLMs with symbolic execution to solve numerical problems. Results show that the method outperforms previous methods on several math datasets. PoT is suitable for problems requiring highly symbolic reasoning skills. CoT may be more effective for semantic reasoning tasks like commonsense reasoning.
- PoT is evaluated on Math word problems (MWP) and financial problems (GSM8k, FinQA..).
- Two limitations: the need to block the LLM from importing additional modules, which may hurt generalization for unknown symbolic tasks; and the difficulty in solving complex algebraic questions using PoT.



Few-shot Performance overview of Codex PoT and Codex CoT across different datasets
14



04 Multi Agent Systems



When Not to use agentic AI

Agents are useful when you need an LLM to determine the workflow of an app.

But they're often overkill. The question is: do I really need flexibility in the workflow to efficiently solve the task at hand? If the pre-determined workflow falls short too often, that means you need more flexibility.

Then think about agentic AI!

But If your workflow is always deterministic and fits all request, just code everything! This will give you a 100% reliable system with no risk of error introduced by letting unpredictable LLMs meddle in your workflow.

For the sake of simplicity and robustness, it's advised to regularize towards not using any agentic behaviour.

<https://huggingface.co/blog/smolagents>





PydanticAI – <https://ai.pydantic.dev>

PydanticAI is a Python agent framework designed to make it less painful to build production grade applications with Generative AI.

The Agent as a container for:

- A system prompt — a set of instructions for the LLM written by the developer
- One or more function tool — functions that the LLM may call to get information while generating a response
- An optional structured result type — the structured datatype the LLM must return at the end of a run
- A dependency type constraint — system prompt functions, tools and result validators may all use dependencies when they're run

Agents may optionally also have a default LLM model associated with them; the model to use can also be specified when running the agent

PydanticAI is an asynchronous library



Huggingface Smolagents

Definition:

1. AI Agents are programs where LLM outputs control the workflow.
2. Any system leveraging LLMs will integrate the LLM outputs into code.
3. The influence of the LLM's input on the code workflow is the level of agency of LLMs in the system.

Note: "agent" is not a discrete, 0 or 1 definition: instead, "agency" evolves on a continuous spectrum, as you give more or less power to the LLM on your workflow.

Introducing *smolagents*, a simple library to build agents

Published December 31, 2024

[Update on GitHub](#)



[m-ric](#)
Aymeric Roucher



[merve](#)
Merve Noyan



[thomwolf](#)
Thomas Wolf

Today we are launching [smolagents](#), a very simple library that unlocks agentic capabilities for language models. Here's a glimpse:

```
from smolagents import CodeAgent, DuckDuckGoSearchTool, HfApiModel

agent = CodeAgent(tools=[DuckDuckGoSearchTool()], model=HfApiModel())

agent.run("How many seconds would it take for a leopard at full speed to run thr
```

<https://huggingface.co/blog/smolagents>



Huggingface Smolagents

Agency Level	Description	How that's called	Example Pattern
☆☆☆	LLM output has no impact on program flow	Simple processor	<code>process_llm_output(llm_response)</code>
★☆☆	LLM output determines basic control flow	Router	<code>if llm_decision(): path_a() else: path_b()</code>
★★☆	LLM output determines function execution	Tool call	<code>run_function(llm_chosen_tool, llm_chosen_args)</code>
★★★	LLM output controls iteration and program continuation	Multi-step Agent	<code>while llm_should_continue(): execute_next_step()</code>
★★★★	One agentic workflow can start another agentic workflow	Multi-Agent	<code>if llm_trigger(): execute_agent()</code>

<https://huggingface.co/blog/smolagents>



Multi-step Agent

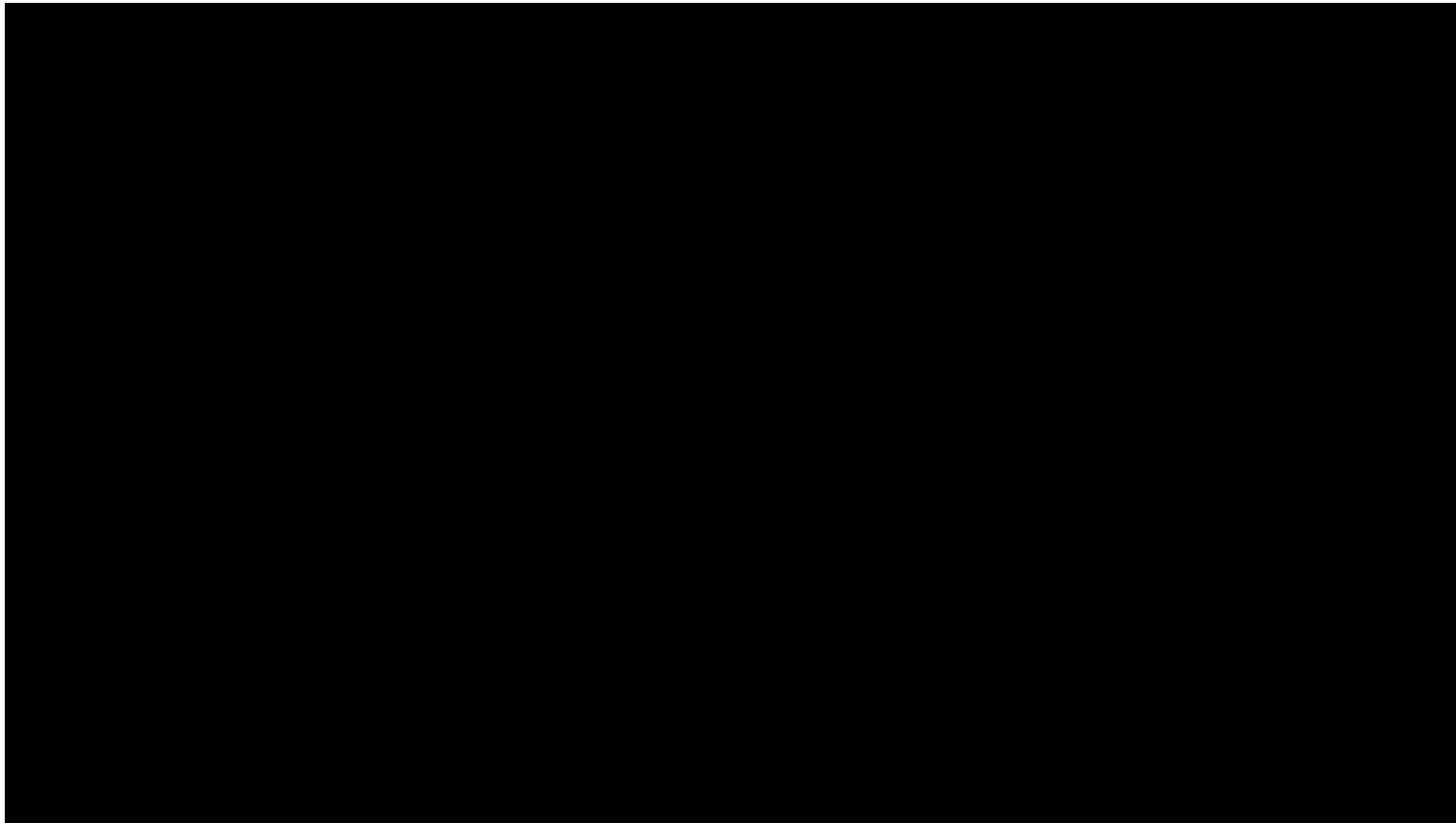
```
memory = [user_defined_task]
while llm_should_continue(memory):
    action = llm_get_next_action(memory)
    observations = execute_action(action)
    memory += [action, observations]
```

```
# this loop is the multi-step part
# this is the tool-calling part
```

<https://huggingface.co/blog/smolagents>



Multi-step Agent



<https://huggingface.co/blog/smolagents>



Autogen

Autogen: „An agent is an entity that can act on behalf of human intent. It can send messages, receive messages, respond to other agents after taking actions and interact with other agents. „



05 Further Reading and Backup



Further Reading

01 – Intelligent Agents:

- Artificial Intelligence - A Modern Approach (Third Edition), Stuart J. Russell and Peter Norvig (Chapter 2 and 26)
- Wooldridge and Jennings, 1995

02 – Reasoning:

- The Cambridge Handbook of Thinking and Reasoning, 2005

More Literature:

- “Large Language Model based Multi-Agents: A Survey of Progress and Challenges”, v2 19.04.2024, <https://arxiv.org/abs/2402.01680>
- “LLM Powered Autonomous Agents”: <https://lilianweng.github.io/posts/2023-06-23-agent/>



Dominik Neumann
Hochschule Reutlingen, Alteburgstraße 150, 72762 Reutlingen
www.reutlingen-university.de
T. +49 172 9861157
dominik.neumann@reutlingen-university.de
dominik.neumann@exxeta.com

